DOUBTBUDDY

A Project Report

Submitted in partial fulfilment of the

Requirements for the award of the Degree of 2024-2025

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

Submitted by:

Khadija Bhanpuarwala

Seat No.:

Under the esteemed guidance of

DEPARTMENT OF INFORMATION TECHNOLOGY

VIDYAVARDHINI's

ANNASAHEB VARTAK COLLEGE OF ARTS,

KEDARNATH MALHOTRA COLLEGE OF COMMERCE,

E.S. ANDRADES COLLEGE OF SCIENCE

(Affiliated to University of Mumbai)

VASAI ROAD – 401202

MAHARASHTRA

2025-26

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN no.: 2023016401768937 **Roll no.:**

1. **Name of the Student: - Khadija Bhanpurawala**

2. **Title of the Project: - DOUBTBUDDY**

3. **Name of the Guide: - Supriya Mhatre**

4. **Teaching experience of the guide** : - 18 **years**

5. **Is this your first submission: -** Yes ☑ No ☐

Signature of the Student **Signature of the Guide**

Date: **Date:**

Signature of the Coordinator **Date:**

VIDYAVARDHINI's

ANNASAHEB VARTAK COLLEGE OF ARTS,

KEDARNATH MALHOTRA COLLEGE OF COMMERCE, E.S. ANDRADES COLLEGE OF SCIENCE

(Affiliated to University of Mumbai)

VASAI ROAD, MAHARASHTRA – 401202

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled **"DoubtBuddy"**, is bonfire work of **Khadija Bhanpurawala** bearing **Seat No:** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide: Supriya Mhatre **Coordinator:**

Internal Examiner: **External Examiner:**

Date: **College Seal:**

ABSTRACT

DoubtBuddy is an innovative web application developed by **Khadija Bhanpurawala**, designed to transform the way students and learners seek academic help and collaborate in real time.

Utilizing modern web technologies, DoubtBuddy offers an intuitive and interactive platform where users can post their doubts, receive instant responses from peers or mentors, and engage in one-on-one chats or video calls for deeper clarification. The frontend, built with React, delivers a smooth and user-friendly interface, while the backend, powered by Node.js, Express.js, and MongoDB, ensures efficient data handling and real-time communication through Web Socket.

Key features include secure user authentication, intelligent doubt categorization, seamless request-accept workflows, a helper rating system, and a scoreboard to encourage participation. By combining real-time interaction, community-driven support, and a structured doubt-resolution process, DoubtBuddy redefines peer learning and fosters a collaborative academic environment.

ACKNOWLEDGEMENT

I am very much thankful to my teacher staff that they have faith on me and gave me opportunity to work on **"DoubtBuddy**" project to express my professional gratitude. Specially. I am very grateful to my project guide "**Mrs. Supriya Mhatre**" under whom I have worked on this. The tips and proper guidance of mam helped me to complete the project as required. Teachers staff are resourceful and provided full support when I needed. A special thanks to our Head of Department "**Mrs. Sushan Brahmana**" to very helpful. Finally, I wish to thanks my all, it department teachers, colleagues who directly or indirectly helped me in completion of the project. Last but not the least I would thank My family and those also whom I forgot to mention.

DECLARATION

I hereby declare that the project entitled, **"DoubtBuddy"** done at Vartak College, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

Khadija Bhanpurawala

Name & Signature of the Student

INDEX

Chapter	Content	Page No.
1	**Introduction**	**9**
	Background	9
	Objective	9
	Purpose	9
	Scope	9
	Applicability	9
	Achievement	10
	Organization Of Report	10
2	**Survey of Technologies**	**11**
3	**REQUIREMENTS AND ANALYSIS**	**12**
	Problem Definition	12
	Requirements Specification	12
	Planning & Scheduling	13
	Software & Hardware Requirement	16
4	**System Design**	**17**
	Basic Modules	17
	Data Design	18
	procedural Diagrams	20
	Security Issues	23
	Test Cases Design	24
	User Interface Design	26
5	**Implementation and testing**	25

6	**Results and Discussion**	**41**
	Test Resp	41
	User Documentation	42
7	**Conclusion**	**44**
	Conclusion	44
	Limitation of the system	44
	Reference	45

TABLE OF FIGURE

Sr. No.	Contents	Page No.
1	Gantt Chart	14
2	PERT diagram	15
3	ER Diagram	19
4	Class Diagram	20
5	Activity Diagram	21
6	Use Case Diagram	22

1. INTRODUCTION

• Background

In the modern era of digital learning, students often face difficulties in getting timely help with their academic doubts. Traditional classrooms and online resources may not always provide immediate, personalized support. DoubtBuddy was developed to solve this issue by creating a peer-based doubt-solving platform where users can ask questions and get quick help from fellow students or mentors, promoting collaborative and real-time learning.

• Objectives

The primary objective of DoubtBuddy is to provide a convenient and interactive space for students to raise academic queries and connect with others who can help. It focuses on building a supportive community through chat or video interactions. The project also aims to implement a reward system based on feedback and ratings, encouraging participation and quality support. It ensures ease of use, subject variety, and real-time doubt resolution.

• Purpose

The purpose of this project is to bridge the communication gap between students and potential helpers by offering a mobile-based solution where doubts can be solved quickly and effectively. DoubtBuddy promotes peer learning, knowledge sharing, and academic collaboration outside of the traditional classroom setting, improving the overall learning experience.

• Scope

DoubtBuddy is designed to serve school and college students across different academic levels and subjects. The platform supports doubt posting, real-time chat and video assistance, user ratings, and a scoreboard system. It can be expanded to include features like subject tagging, expert recommendations, and group discussions, making it scalable and adaptable to future educational needs.

• Applicability

This project is highly applicable in educational institutions, tutoring communities, and online learning environments. It can be used by students who need academic help, mentors who wish to guide others, and institutions that aim to improve student engagement and performance. It is also relevant during remote learning and exam preparation phases.

• Achievements

Through the development of the DoubtBuddy project, we aim to achieve several key milestones. A functional and user-friendly application will be developed to allow students to register, post academic doubts, and receive real-time support through chat and video calls. A feedback and rating system will be implemented to ensure quality responses and reward active users via a scoreboard. The platform will be tested with a sample user group to evaluate its effectiveness, usability, and performance. These achievements will demonstrate the project's potential to improve academic collaboration and peer-based learning.

• Organization of Report

The report is structured to provide a comprehensive overview of the DoubtBuddy project. The following chapters are organized as follows:

I. Literature Review: An in-depth analysis of existing systems, tools, and platforms related to doubt-solving, peer-to-peer learning, and educational assistance applications.
II. System Design: A detailed description of the overall system architecture, including the design of user interfaces, backend workflows, and database structures that power the core functionalities of DoubtBuddy.
III. Implementation: A step-by-step guide on how the application was developed, highlighting key modules such as user registration, doubt card creation, helper matching, chat/video interaction, and star-rating system.
IV. Testing and Evaluation: The strategies and tools used to test the app's performance, usability, and reliability, along with a summary of the results and areas for improvement.

Conclusion and Future Work: A summary of key outcomes, insights gained during development, and suggestions for future enhancements like AI-based doubt recommendations or gamified learning incentives.

2. ***SURVEY OF TECHNOLOGIES***

Technology	Features
React.js	Frontend of the application
Redux	State Management
Node.js, Express.js	Backend of the application
MongoDB Atlas, Mongoose	Database for the application
Bcrypt	Password Management
JSON Web Token	Authorization and Authentication
Render	Deployment
Postman	API Testing, Debugging and Documentation

3. REQUIREMENTS AND ANALYSIS

• Problem Definition

In today's digital learning environment, students often struggle to find immediate solutions to their academic doubts. Existing platforms such as forums or messaging apps fail to provide real-time doubt resolution and structured collaboration between students and peers. The primary problem being addressed in this project is the lack of an integrated platform where students can ask, discuss, and resolve doubts in real time, with proper authentication, categorization, and user-friendly communication features.

• Requirements specification

The requirements for the DoubtBuddy application are defined independent of implementation specifics.

The following specifications outline the functionalities and operations of the system:

• Functional Requirements:

➢ User Authentication:

Users can register, log in, and update their profiles.

Secure login with email/phone + password/OTP.

Password reset and account recovery options

➢ Question and doubt posting

Students can post doubts in text or image.

Option to edit or delete posted doubts

➢ Real-Time collaboration

Live chat between users

Support for voice/video sessions.

➢ Answering and discussion

peers can provide answers with text, images, or links.

Commenting and discussion threads under each doubt.

• Non-Functional Requirements

➢ Performance:

o the application should support real-time editing with minimal latency.

➢ Scalability:

o the system must handle a growing number of users and doubts without degrading performance.

➢ Security: o User data should be encrypted and protected against

unauthorized access.

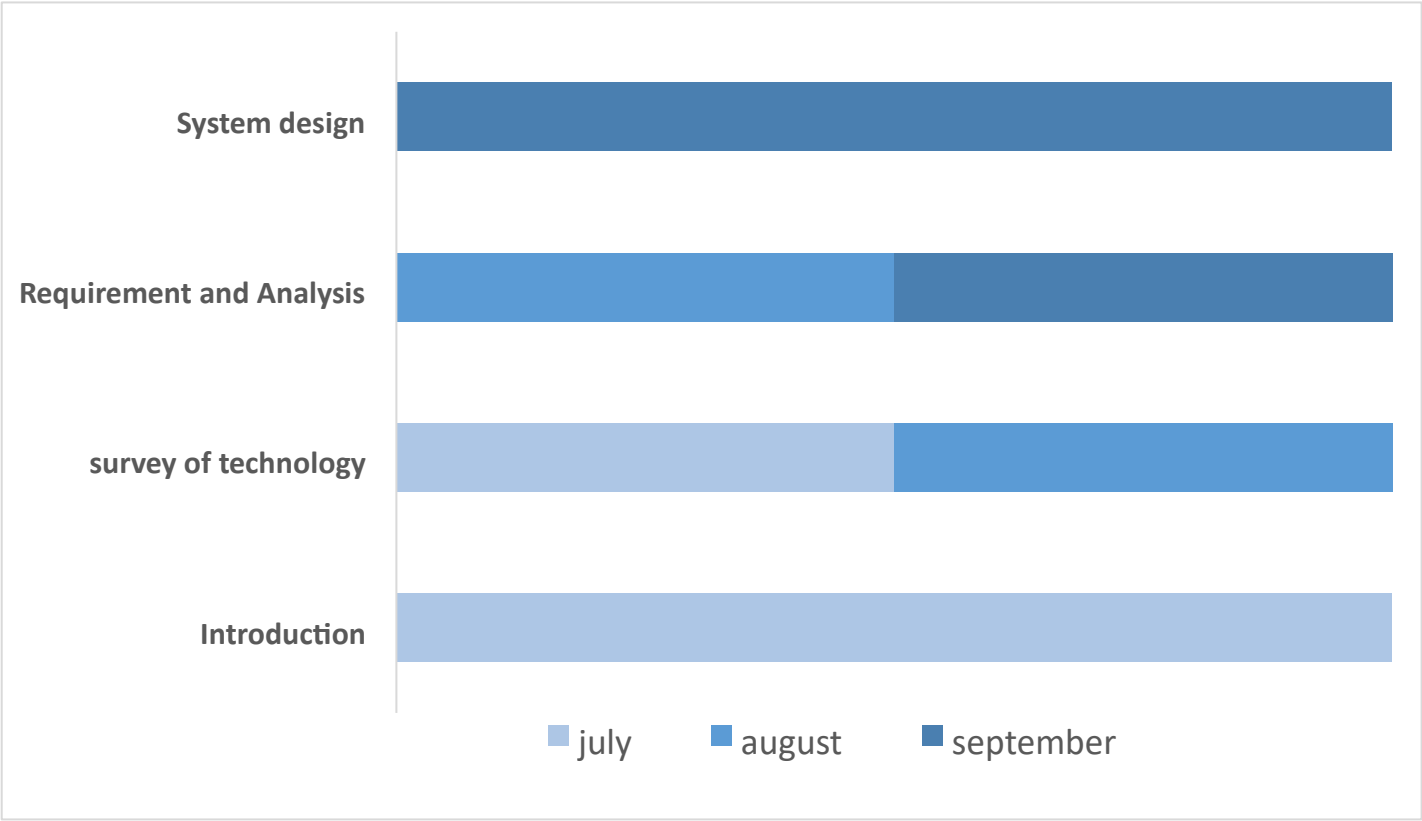
• Planning and Scheduling

Effective planning and scheduling are critical for the successful completion of the project.

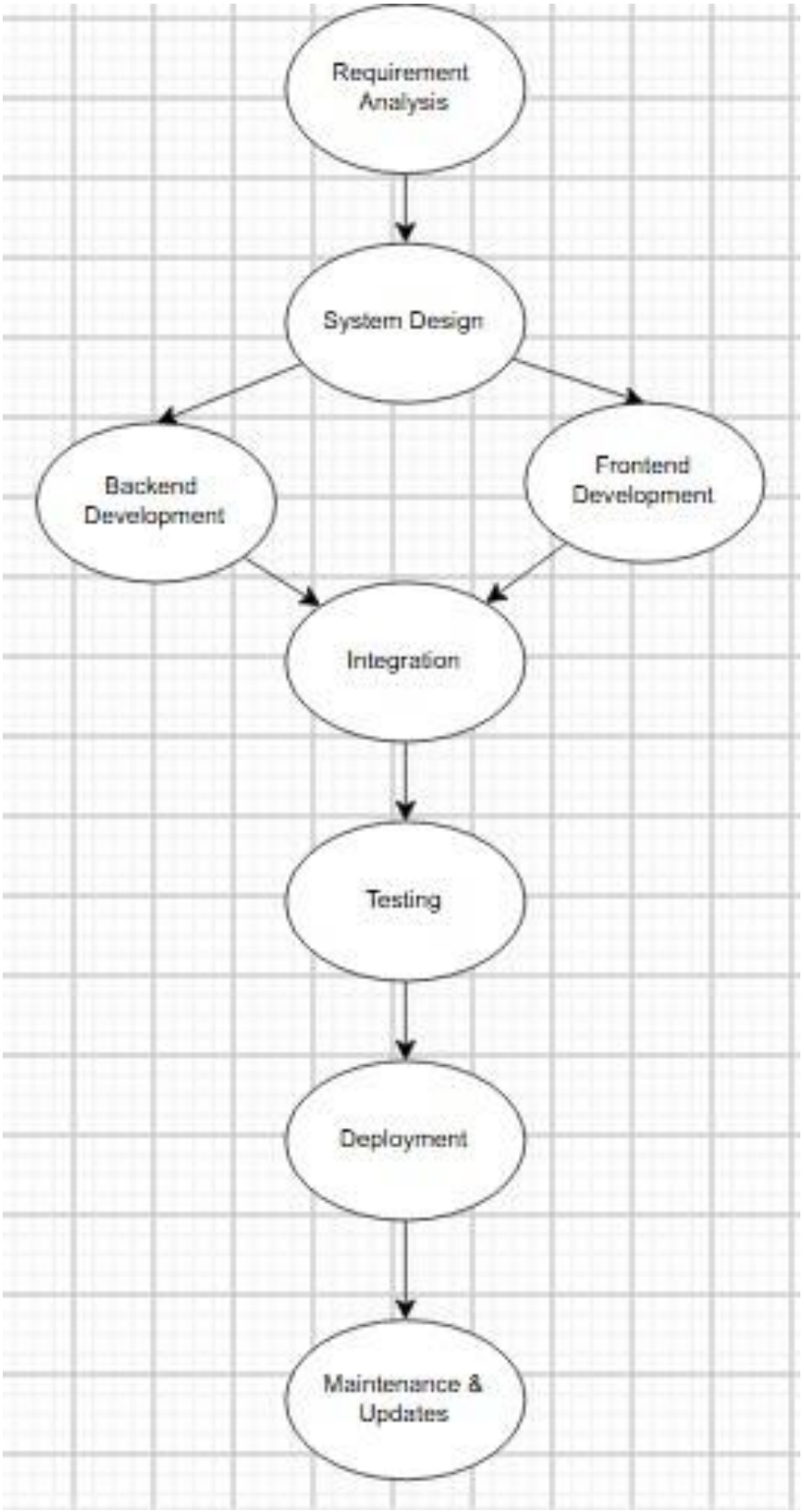
The planning phase involves breaking down the project into smaller, manageable tasks, while the

scheduling phase ensures that adequate resources are available to accomplish these tasks

• Gantt Chart


System design
Requirement and Analysis
survey of technology
Introduction
july
august
september

- PERT DAIGRAM



• Software and Hardware Requirements

➢ Hardware Requirements

- **Developer Machine**: i5/Ryzen 5, 8 GB RAM (16 GB preferred), 256 GB SSD, Windows/Linux/macOS.
- **Server (Small scale):** 2 vCPU, 4 GB RAM, 40 GB SSD. ○ **Server (Medium scale):** 4 vCPU, 8–16 GB RAM, 100+ GB SSD.
- **Client Device:** Any modern smartphone or PC with browser, 2 GB RAM minimum.

➢ Software Requirements:

○ **Frontend:** HTML, CSS, JS, React.js (or Flutter for app). ○ **Backend:** Python (Django/Flask/FastAPI) or Node.js (Express). ○ **Database:** MongoDB / MySQL / PostgreSQL. ○ **Authentication:** JWT / OAuth2. ○ **Real-time:** WebSockets / Socket.IO. ○ **Hosting:** AWS / Google Cloud / DigitalOcean.

4. ***System Design***

• Basic Modules

The DoubtBuddy project adopts a modular approach to simplify development by dividing the system into manageable parts. Each module can be developed independently and later integrated into a unified system.

○ Authentication

Ensures users (students, mentors, or admins) can securely register, log in, and access the platform. Supports role-based access (e.g., student, mentor**).**

○ Doubt Management

Allows students to post their doubts with text, images, or code snippets.

Provides mentors the ability to answer, update, and resolve doubts.

Includes tagging and categorization of doubts for easy search.

○ Real-Time Interaction

Enables instant chat, live Q&A, or video call features for resolving doubts.

Uses WebSocket for real-time communication**.**

○ Collaboration & Discussion

Supports group discussions where multiple users can contribute to solving a doubt.

Implements voting/like system for best answers.

○ Sharing & Notification

Provides functionality for sharing doubt links with peers or mentors.

Sends real-time notifications when a new answer, comment, or update is posted.

• Data Design

Data design consist of how data is organised, managed and manipulate.

○ Data Requirements

What kind of data will be stored (users, doubts, answers, chats, etc.) Data input/output formats (text, images, files).

Data security requirements (encryption, access control)

○ Data Entities & Attributes

List the main entities and their key attributes. Example for DoubtBuddy:

User: user_id, name, email, password_hash, role (student/teacher/admin), join_date.

Doubt: doubt_id, user_id, subject, topic, description, attachment, timestamp, status.

Answer: answer_id, doubt_id, user_id, answer_text, attachment, votes, timestamp.

Discussion/Comment: comment_id, answer_id, user_id, text, timestamp.

Collaboration (Chat/Session): chat_id, user_id(s), message, timestamp, type (text/voice/video).

○ Relationships (ER Model)

One user can post many doubts.

One doubt can have many answers.

One answer can have many comments.

Many users can participate in one collaboration session.

○ Data Flow / Access Methods

How data moves between modules (e.g., when a doubt is posted → stored in database → notification sent → accessible to others).

CRUD operations (Create, Read, Update, Delete) for each entity.

○ Database Schema (Logical Design)

Define tables, primary keys, foreign keys.

Example:

User(user_id PK, name, email, password_hash, role)

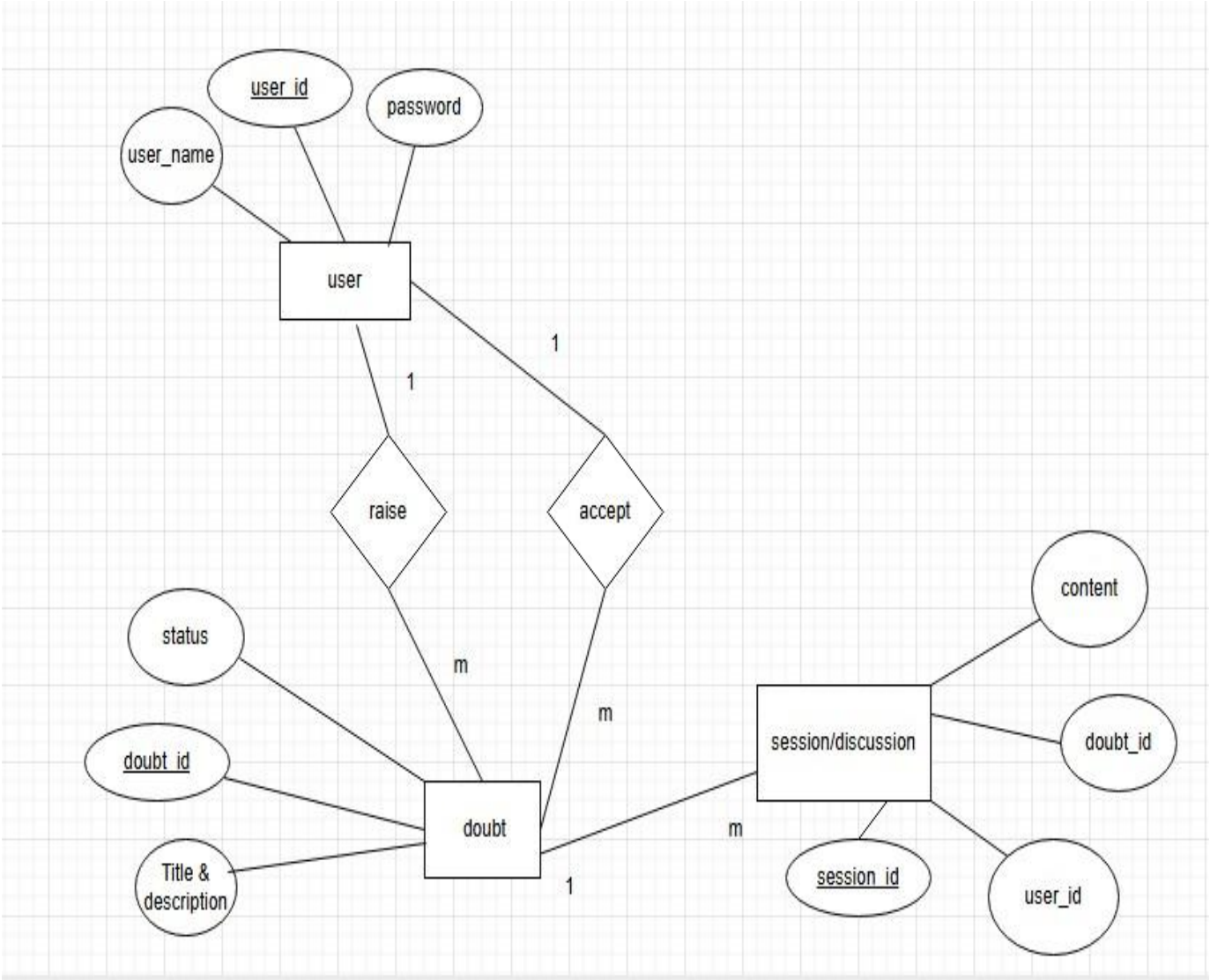
Doubt(doubt_id PK, user_id FK, subject, description, status)

Answer(answer_id PK, doubt_id FK, user_id FK, answer_text, votes)

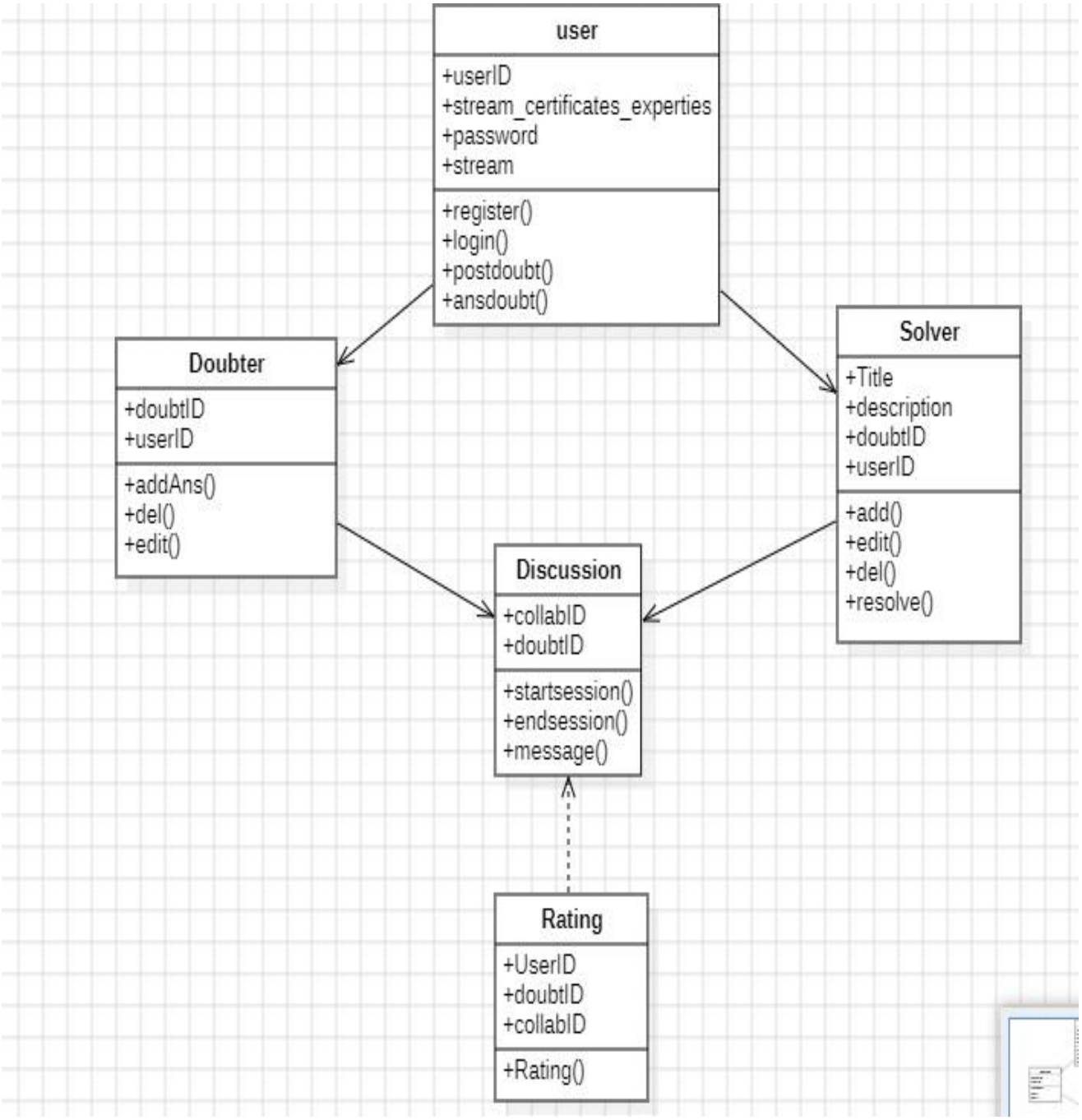
- Procedural Design

Procedural Design is a method used in **software engineering** and **system design** that focuses on developing a system through **step-by-step procedures** or routines. It breaks down a system into smaller, manageable **functions or procedures**, which are then organized in a **hierarchical** structure to perform specific tasks.

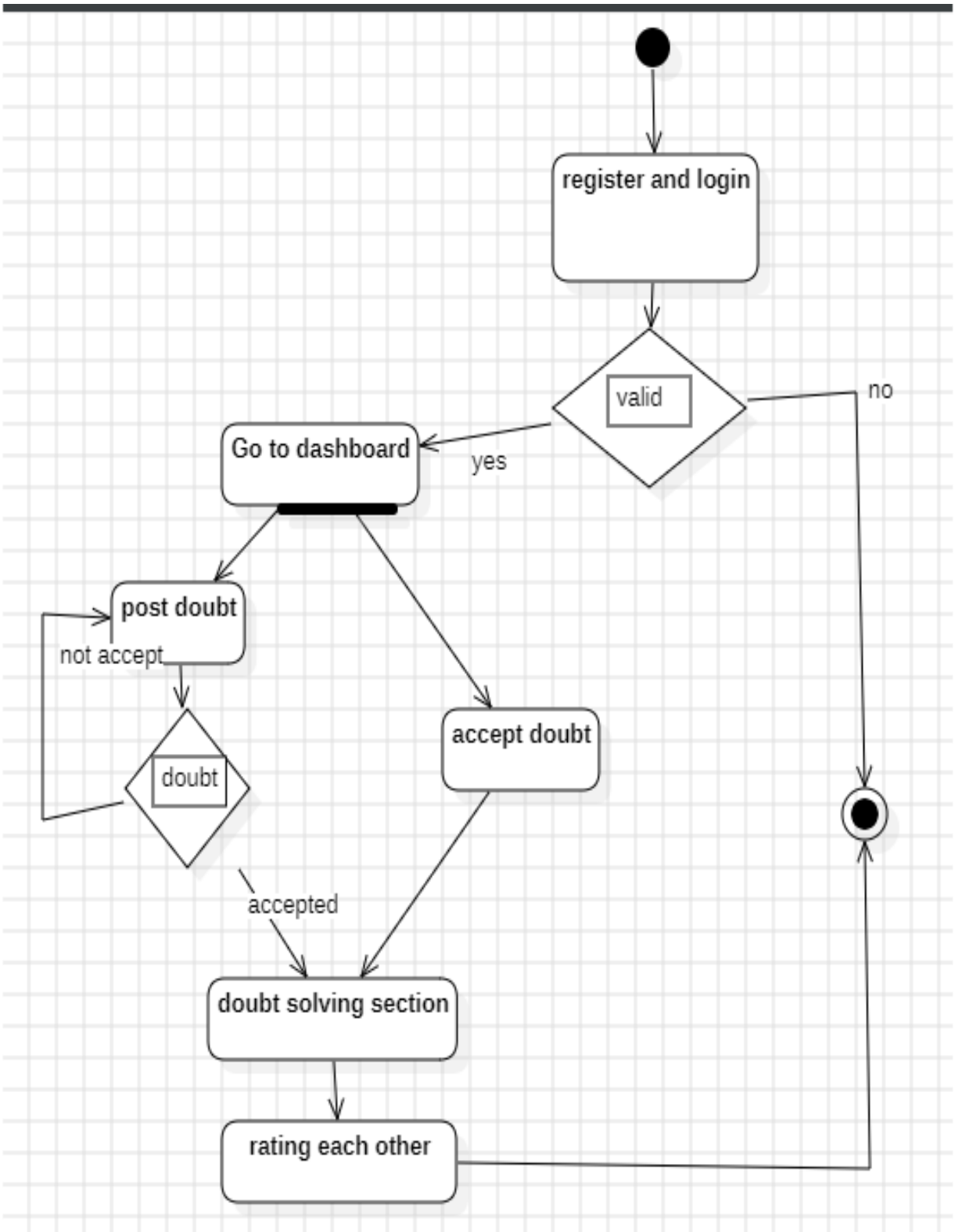
ER Diagram



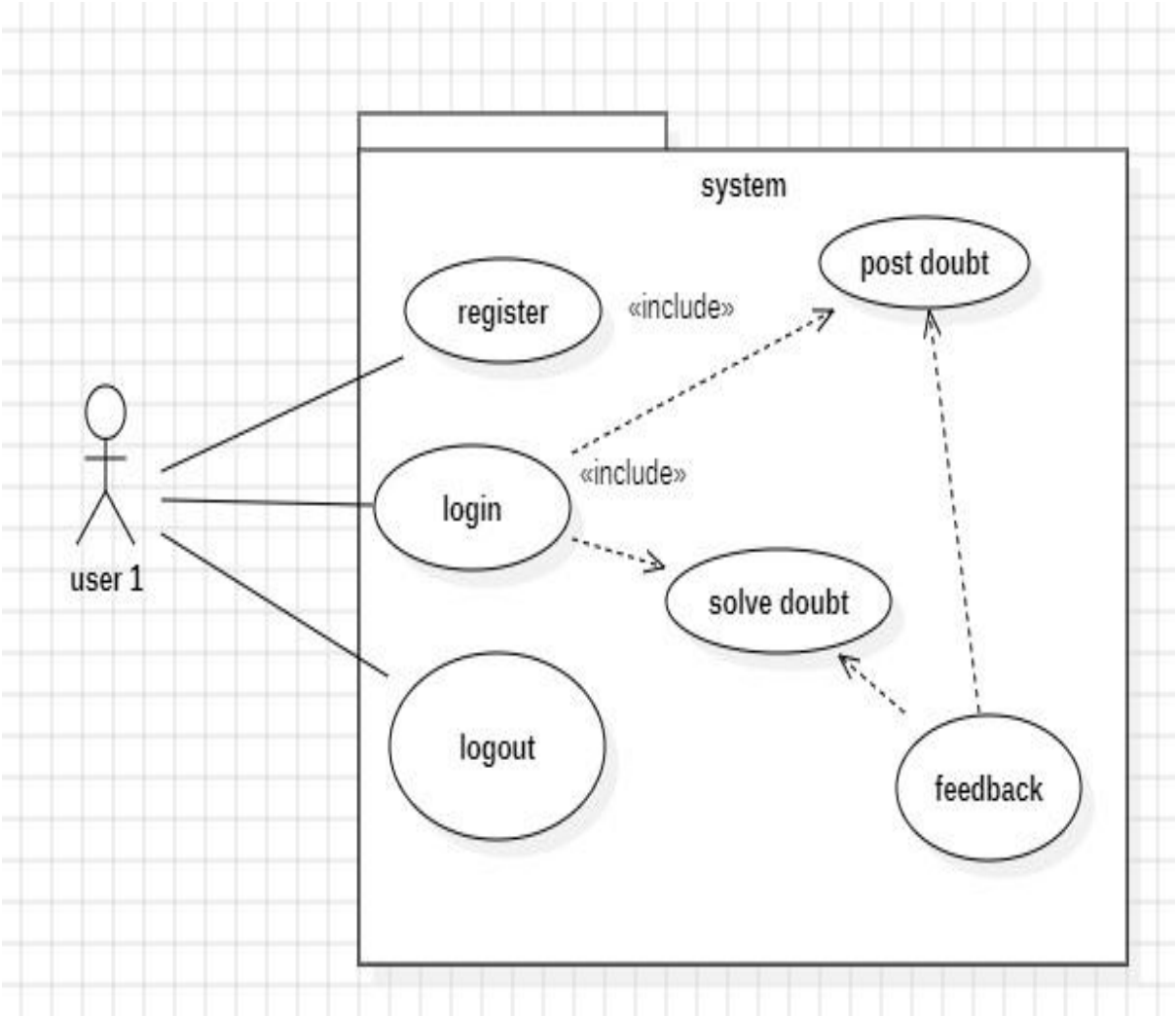
- Class Diagram



- Activity Diagram



- USECASE DAIGRAM



• Security Issues in DoubtBuddy

- Weak Authentication – Easy passwords, no MFA.
- Data Privacy – Leaks of student/teacher info, no encryption.
- Session Hijacking – Unauthorized users joining doubt sessions.
- Injection Attacks – SQL injection, XSS through chat/inputs.
- File Upload Risks – Malware disguised as notes/images.
- DDoS Attacks – System overload, no rate limiting.
- Impersonation – Fake teacher/student accounts.

- Test case Design

Module	Test case	Expected result
Authentication	Login with valid credentials	User logs in successfully
	Login with invalid credentials	Error message shown
	Register with existing email	System rejects registration
Ask Doubt	Post a doubt with valid input	Doubt appears in list
	Post a doubt with empty field	Error message shown
Answer Doubt	Helper/user answers a doubt	Answer linked to doubt
Chat/Live Session	User joins live session	Connection established
Logout	User clicks logout	Session ends, redirected to login

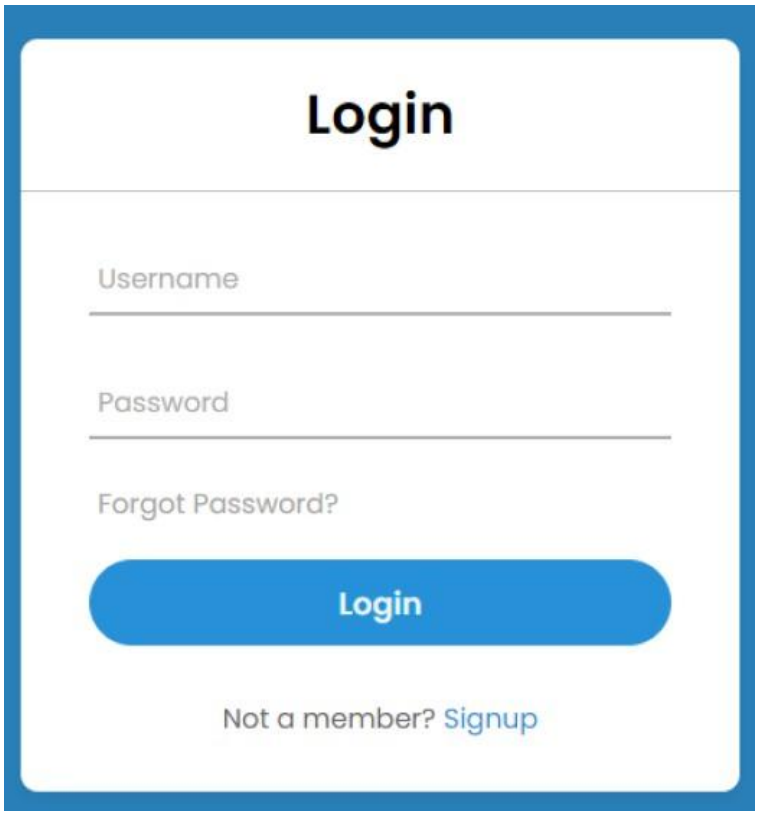
- User interface

A **user interface (UI)** is the point of interaction between a user and a digital device or software application. It includes everything the user sees and interacts with, such as buttons, icons, menus, text, images, and other visual elements. The main goal of a user interface is to make it easy, efficient, and enjoyable for users to navigate and use

a system. A well-designed UI is intuitive, meaning users can understand and use it without needing extensive instructions. Whether on a smartphone, website, or computer program, the user interface plays a critical role in the overall user experience (UX)

.

Logic page:

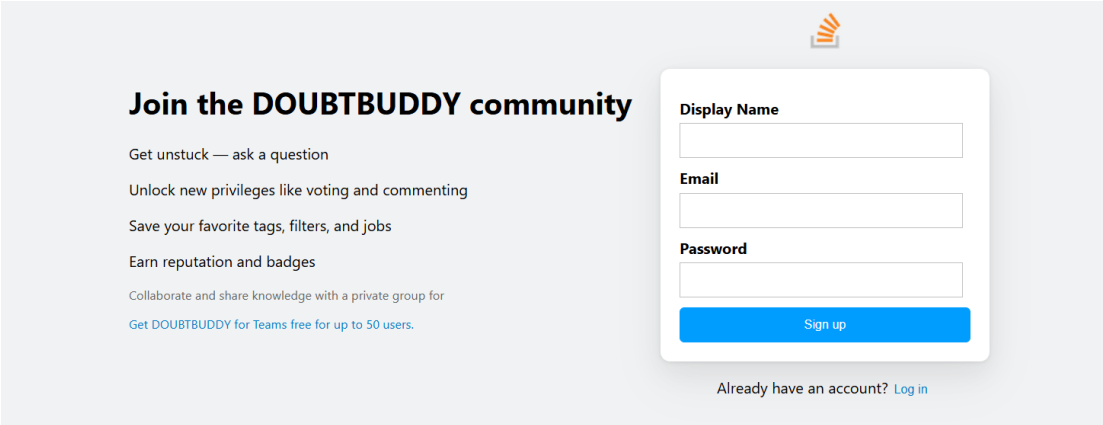


5. *Implementation and Testing*

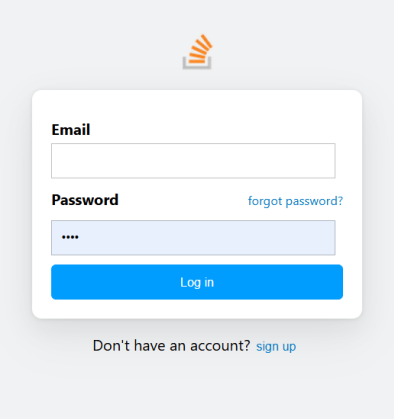
- Auth page

```
import jwt from "jsonwebtoken";
import bcrypt from "bcryptjs";

import users from "../models/auth.js";

export const signup = async (req, res) => {
  const { name, email, password } = req.body;
  try {
    const existinguser = await users.findOne({ email });
    if (existinguser) {
      return res.status(404).json({ message: "User already Exist." });
    }

    const hashedPassword = await bcrypt.hash(password, 12);
    const newUser = await users.create({
      name,
      email,
      password: hashedPassword,
    });
    const token = jwt.sign(
      { email: newUser.email, id: newUser._id },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );
    res.status(200).json({ result: newUser, token });
  } catch (error) {
    res.status(500).json("Something went worng...");
  }
};

export const login = async (req, res) => {
  const { email, password } = req.body;
  try {
    const existinguser = await users.findOne({ email });
    if (!existinguser) {
      return res.status(404).json({ message: "User don't Exist." });
    }
    const isPasswordCrt = await bcrypt.compare(password, existinguser.password);
    if (!isPasswordCrt) {
      return res.status(400).json({ message: "Invalid credentials" });
    }
    const token = jwt.sign(
      { email: existinguser.email, id: existinguser._id },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );
    res.status(200).json({ result: existinguser, token });
  } catch (error) {
```

```
    res.status(500).json("Something went worng...");
  }
};
```



-

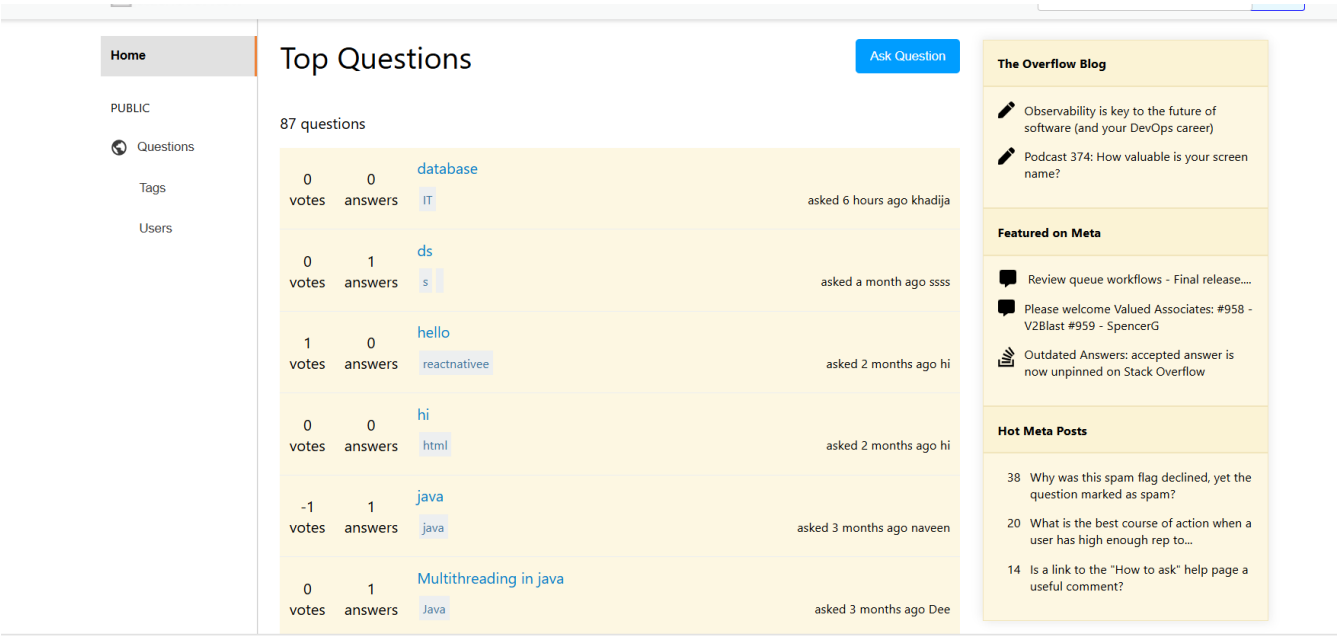


module.exports = mongoose.model("User", userSchema);

- HOME PAGE

```
import React from "react";

import "../../App.css";
import LeftSidebar from "../../components/LeftSidebar/LeftSidebar";
import RightSidebar from "../../components/RightSidebar/RightSidebar";
import HomeMainbar from "../../components/HomeMainbar/HomeMainbar";

const Home = ({ slideIn }) => {
  return (
    <div className="home-container-1">
      <LeftSidebar slideIn={slideIn} />
      <div className="home-container-2">
        <HomeMainbar />
        <RightSidebar />
      </div>
    </div>
  );
};
```

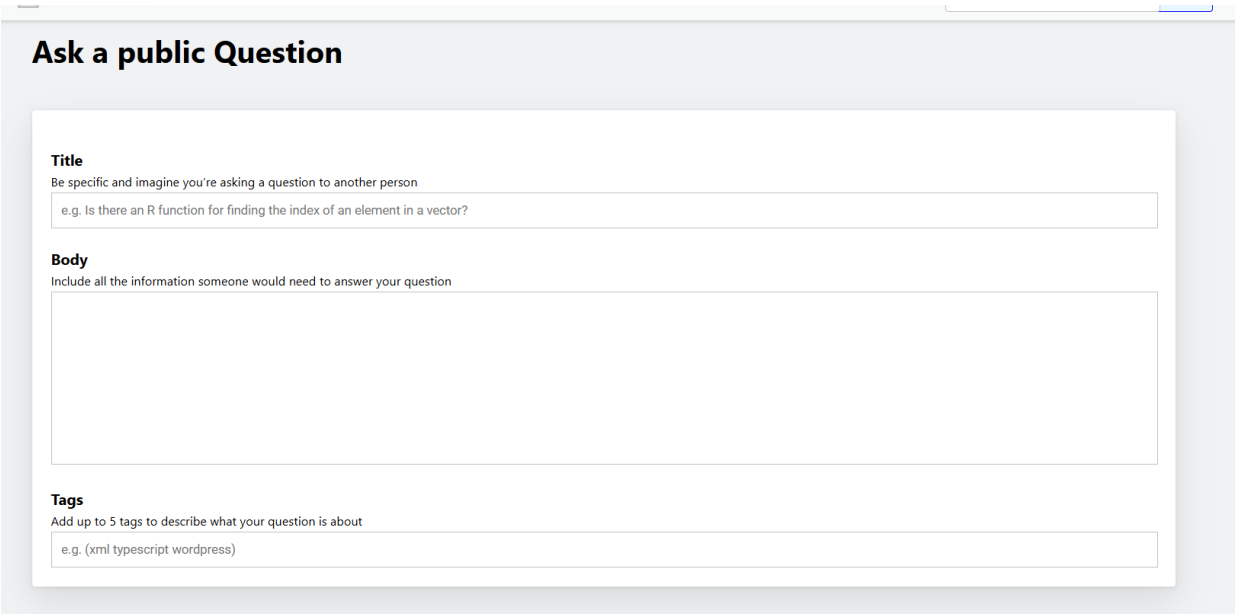
```
export default Home;
```



- QUESTION PAGE

```
import * as api from "../api/index";

export const askQuestion = (questionData, navigate) => async (dispatch) => {
  try {
    const { data } = await api.postQuestion(questionData);
    dispatch({ type: "POST_QUESTION", payload: data });
    dispatch(fetchAllQuestions());
    navigate("/");
  } catch (error) {
    console.log(error);
  }
};

export const fetchAllQuestions = () => async (disptach) => {
  try {
    const { data } = await api.getAllQuestions();
    disptach({ type: "FETCH_ALL_QUESTIONS", payload: data });
  } catch (error) {
    console.log(error);
  }
};

export const deleteQuestion = (id, navigate) => async (dispatch) => {
  try {
    await api.deleteQuestion(id);
    dispatch(fetchAllQuestions());
    navigate("/");
  } catch (error) {
    console.log(error);
  }
};
```

```
export const voteQuestion = (id, value) => async (dispatch) => {
  try {
    await api.voteQuestion(id, value);
    dispatch(fetchAllQuestions());
  } catch (error) {
    console.log(error);
  }
};

export const postAnswer = (answerData) => async (dispatch) => {
  try {
    const { id, noOfAnswers, answerBody, userAnswered } = answerData;
    const { data } = await api.postAnswer(
      id,
      noOfAnswers,
      answerBody,
      userAnswered
    );
    dispatch({ type: "POST_ANSWER", payload: data });
    dispatch(fetchAllQuestions());
  } catch (error) {
    console.log(error);
  }
};

export const deleteAnswer = (id, answerId, noOfAnswers) => async (dispatch) => {
  try {
    await api.deleteAnswer(id, answerId, noOfAnswers);
    dispatch(fetchAllQuestions());
  } catch (error) {
    console.log(error);
  }
};
```



- Question detail

```
import React, { useState } from "react";
import { useParams, Link, useNavigate, useLocation } from "react-router-dom";
import { useSelector, useDispatch } from "react-redux";
import moment from "moment";
import copy from "copy-to-clipboard";

import upvote from "../../assets/sort-up.svg";
import downvote from "../../assets/sort-down.svg";
import "./Questions.css";
import Avatar from "../../components/Avatar/Avatar";
import DisplayAnswer from "./DisplayAnswer";
import {
  postAnswer,
  deleteQuestion,
  voteQuestion,
} from "../../actions/question";

const QuestionsDetails = () => {
  const { id } = useParams();
  const questionsList = useSelector((state) => state.questionsReducer);

  const [Answer, setAnswer] = useState("");
  const Navigate = useNavigate();
  const dispatch = useDispatch();
  const User = useSelector((state) => state.currentUserReducer);
  const location = useLocation();
  const url = "http://localhost:3000";

  const handlePostAns = (e, answerLength) => {
    e.preventDefault();
    if (User === null) {
      alert("Login or Signup to answer a question");
      Navigate("/Auth");
    } else {
      if (Answer === "") {
        alert("Enter an answer before submitting");
      } else {
        dispatch(
          postAnswer({
            id,
            noOfAnswers: answerLength + 1,
            answerBody: Answer,
            userAnswered: User.result.name,
          })
        );
        setAnswer("");
      }
    }
  };

  const handleShare = () => {
    copy(url + location.pathname);
    alert("Copied url : " + url + location.pathname);
```

```
  };

  const handleDelete = () => {
    dispatch(deleteQuestion(id, Navigate));
  };

  const handleUpVote = () => {
    if (User === null) {
      alert("Login or Signup to up vote a question");
      Navigate("/Auth");
    } else {
      dispatch(voteQuestion(id, "upVote"));
    }
  };

  const handleDownVote = () => {
    if (User === null) {
      alert("Login or Signup to down vote a question");
      Navigate("/Auth");
    } else {
      dispatch(voteQuestion(id, "downVote"));
    }
  };

  return (
    <div className="question-details-page">
      {questionsList.data === null ? (
        <h1>Loading...</h1>
      ) : (
        <>
          {questionsList.data
            .filter((question) => question._id === id)
            .map((question) => (
              <div key={question._id}>
                <section className="question-details-container">
                  <h1>{question.questionTitle}</h1>
                  <div className="question-details-container-2">
                    <div className="question-votes">
                      <img
                        src={upvote}
                        alt=""
                        width="18"
                        className="votes-icon"
                        onClick={handleUpVote}
                      />
                      <p>{question.upVote.length - question.downVote.length}</p>
                      <img
                        src={downvote}
                        alt=""
                        width="18"
                        className="votes-icon"
                        onClick={handleDownVote}
                      />
                    </div>
                    <div style={{ width: "100%" }}>
```

```
        <p className="question-body">{question.questionBody}</p>
        <div className="question-details-tags">
          {question.questionTags.map((tag) => (
            <p key={tag}>{tag}</p>
          ))}
        </div>
        <div className="question-actions-user">
          <div>
            <button type="button" onClick={handleShare}>
              Share
            </button>
            {User?.result?._id === question?.userId && (
              <button type="button" onClick={handleDelete}>
                Delete
              </button>
            )}
          </div>
          <div>
            <p>asked {moment(question.askedOn).fromNow()}</p>
            <Link
              to={`/Users/${question.userId}`}
              className="user-link"
              style={{ color: "#0086d8" }}
            >
              <Avatar
                backgroundColor="orange"
                px="8px"
                py="5px"
                borderRadius="4px"
              >
                {question.userPosted.charAt(0).toUpperCase()}
              </Avatar>
              <div>{question.userPosted}</div>
            </Link>
          </div>
        </div>
      </div>
    </div>
  </section>
  {question.noOfAnswers !== 0 && (
    <section>
      <h3>{question.noOfAnswers} Answers</h3>
      <DisplayAnswer
        key={question._id}
        question={question}
        handleShare={handleShare}
      />
    </section>
  )}
  <section className="post-ans-container">
    <h3>Your Answer</h3>
    <form
      onSubmit={(e) => {
        handlePostAns(e, question.answer.length);
      }}
```

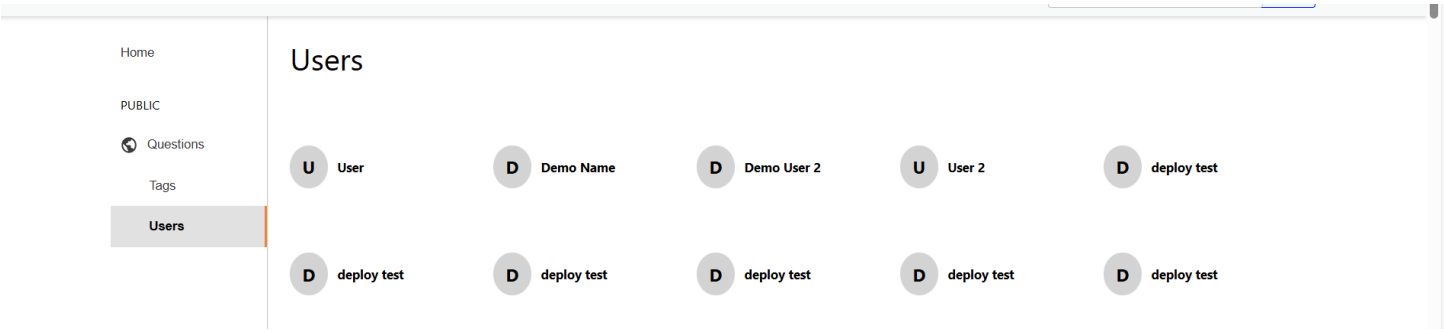
```
                >
                  <textarea
                    name=""
                    id=""
                    cols="30"
                    rows="10"
                    value={Answer}
                    onChange={(e) => setAnswer(e.target.value)}
                  ></textarea>
                  <br />
                  <input
                    type="submit"
                    className="post-ans-btn"
                    value="Post Your Answer"
                  />
                </form>
                <p>
                  Browse other Question tagged
                  {question.questionTags.map((tag) => (
                    <Link to="/Tags" key={tag} className="ans-tags">
                      {" "}
                      {tag}{" "}
                    </Link>
                  ))}{" "}
                  or
                  <Link
                    to="/AskQuestion"
                    style={{ textDecoration: "none", color: "#009dff" }}
                  >
                    {" "}
                    ask your own question.
                  </Link>
                </p>
              </section>
            </div>
          ))}
        </>
      )}
    </div>
  );
};

export default QuestionsDetails;
```

- Answer section
- User profile page

```
import React, { useState } from "react";
import { useSelector } from "react-redux";
import { useParams } from "react-router";
import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
import { faBirthdayCake, faPen } from "@fortawesome/free-solid-svg-icons";
import moment from "moment";

import LeftSidebar from "../../components/LeftSidebar/LeftSidebar";
import Avatar from "../../components/Avatar/Avatar";
import EditProfileForm from "./EditProfileForm";
```

```
import ProfileBio from "./ProfileBio";
import "./UsersProfile.css";

const UserProfile = ({ slideIn, handleSlideIn }) => {
  const { id } = useParams();
  const users = useSelector((state) => state.usersReducer);
  const currentProfile = users.filter((user) => user._id === id)[0];
  const currentUser = useSelector((state) => state.currentUserReducer);
  const [Switch, setSwitch] = useState(false);

  return (
    <div className="home-container-1">
      <LeftSidebar slideIn={slideIn} handleSlideIn={handleSlideIn} />
      <div className="home-container-2">
        <section>
          <div className="user-details-container">
            <div className="user-details">
              <Avatar
                backgroundColor="purple"
                color="white"
                fontSize="50px"
                px="40px"
                py="30px"
              >
                {currentProfile?.name.charAt(0).toUpperCase()}
              </Avatar>
              <div className="user-name">
                <h1>{currentProfile?.name}</h1>
                <p>
                  <FontAwesomeIcon icon={faBirthdayCake} /> Joined{" "}
                  {moment(currentProfile?.joinedOn).fromNow()}
                </p>
              </div>
            </div>
            {currentUser?.result._id === id && (
              <button
                type="button"
                onClick={() => setSwitch(true)}
                className="edit-profile-btn"
              >
                <FontAwesomeIcon icon={faPen} /> Edit Profile
              </button>
            )}
          </div>
          <>
            {Switch ? (
              <EditProfileForm
                currentUser={currentUser}
                setSwitch={setSwitch}
              />
            ) : (
              <ProfileBio currentProfile={currentProfile} />
            )}
          </>
        </section>
```

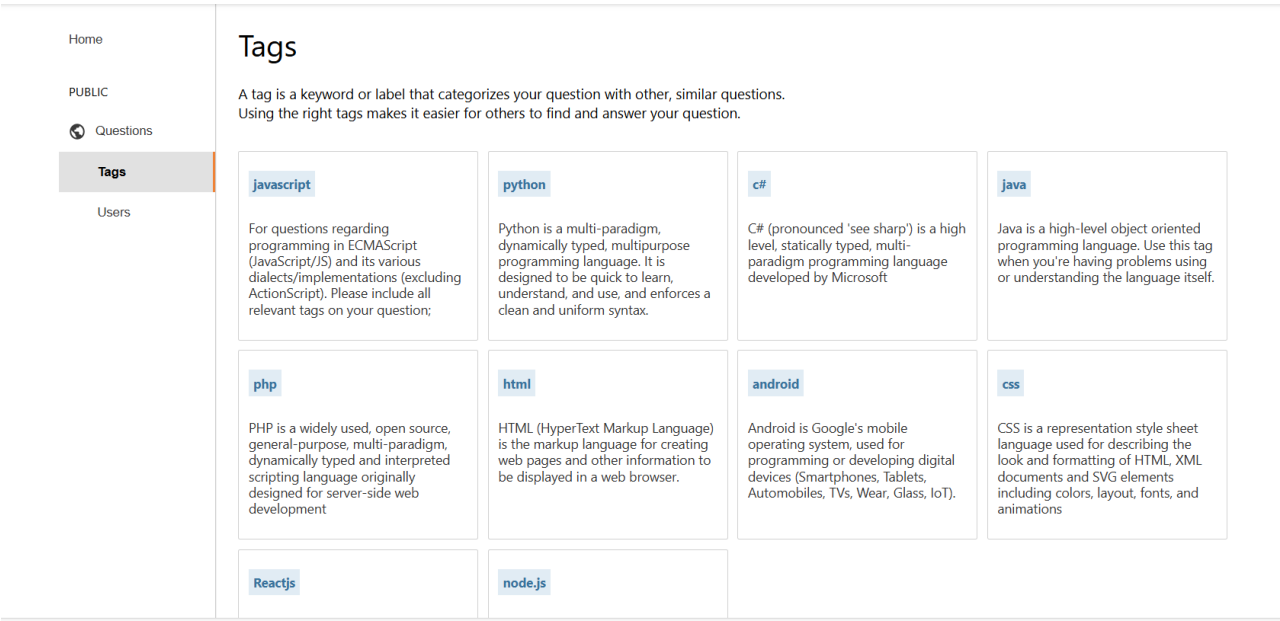
```
        </div>
      </div>
    );
  };

  export default UserProfile;
```



- Tag page

```
import React from "react";

import LeftSidebar from "../../components/LeftSidebar/LeftSidebar";
import TagsList from "./TagsList";
import "./Tags.css";
import { tagsList } from "./tagList";

const Tags = ({ slideIn, handleSlideIn }) => {
  return (
    <div className="home-container-1">
      <LeftSidebar slideIn={slideIn} handleSlideIn={handleSlideIn} />
      <div className="home-container-2">
        <h1 className="tags-h1">Tags</h1>
        <p className="tags-p">
          A tag is a keyword or label that categorizes your question with other,
          similar questions.
        </p>
        <p className="tags-p">
          Using the right tags makes it easier for others to find and answer
          your question.
        </p>
        <div className="tags-list-container">
          {tagsList.map((tag, index) => (
            <TagsList tag={tag} key={index} />
          ))}
```

```
        </div>
      </div>
    </div>
  );
};

export default Tags;
```



- Queston detail

```
   import React, { useState } from "react";
import { useParams, Link, useNavigate, useLocation } from "react-router-dom";
import { useSelector, useDispatch } from "react-redux";
import moment from "moment";
import copy from "copy-to-clipboard";

import upvote from "../../assets/sort-up.svg";
import downvote from "../../assets/sort-down.svg";
import "./Questions.css";
import Avatar from "../../components/Avatar/Avatar";
import DisplayAnswer from "./DisplayAnswer";
import {
  postAnswer,
  deleteQuestion,
  voteQuestion,
} from "../../actions/question";

const QuestionsDetails = () => {
  const { id } = useParams();
  const questionsList = useSelector((state) => state.questionsReducer);

  const [Answer, setAnswer] = useState("");
  const Navigate = useNavigate();
  const dispatch = useDispatch();
  const User = useSelector((state) => state.currentUserReducer);
  const location = useLocation();
```

```
const url = "http://localhost:3000";

const handlePostAns = (e, answerLength) => {
  e.preventDefault();
  if (User === null) {
    alert("Login or Signup to answer a question");
    Navigate("/Auth");
  } else {
    if (Answer === "") {
      alert("Enter an answer before submitting");
    } else {
      dispatch(
        postAnswer({
          id,
          noOfAnswers: answerLength + 1,
          answerBody: Answer,
          userAnswered: User.result.name,
        })
      );
      setAnswer("");
    }
  }
};

const handleShare = () => {
  copy(url + location.pathname);
  alert("Copied url : " + url + location.pathname);
};

const handleDelete = () => {
  dispatch(deleteQuestion(id, Navigate));
};

const handleUpVote = () => {
  if (User === null) {
    alert("Login or Signup to up vote a question");
    Navigate("/Auth");
  } else {
    dispatch(voteQuestion(id, "upVote"));
  }
};

const handleDownVote = () => {
  if (User === null) {
    alert("Login or Signup to down vote a question");
    Navigate("/Auth");
  } else {
    dispatch(voteQuestion(id, "downVote"));
  }
};

return (
  <div className="question-details-page">
    {questionsList.data === null ? (
      <h1>Loading...</h1>
```

```
) : (
  <>
    {questionsList.data
      .filter((question) => question._id === id)
      .map((question) => (
        <div key={question._id}>
          <section className="question-details-container">
            <h1>{question.questionTitle}</h1>
            <div className="question-details-container-2">
              <div className="question-votes">
                <img
                  src={upvote}
                  alt=""
                  width="18"
                  className="votes-icon"
                  onClick={handleUpVote}
                />
                <p>{question.upVote.length - question.downVote.length}</p>
                <img
                  src={downvote}
                  alt=""
                  width="18"
                  className="votes-icon"
                  onClick={handleDownVote}
                />
              </div>
              <div style={{ width: "100%" }}>
                <p className="question-body">{question.questionBody}</p>
                <div className="question-details-tags">
                  {question.questionTags.map((tag) => (
                    <p key={tag}>{tag}</p>
                  ))}
                </div>
                <div className="question-actions-user">
                  <div>
                    <button type="button" onClick={handleShare}>
                      Share
                    </button>
                    {User?.result?._id === question?.userId && (
                      <button type="button" onClick={handleDelete}>
                        Delete
                      </button>
                    )}
                  </div>
                  <div>
                    <p>asked {moment(question.askedOn).fromNow()}</p>
                    <Link
                      to={`/Users/${question.userId}`}
                      className="user-link"
                      style={{ color: "#0086d8" }}
                    >
                      <Avatar
                        backgroundColor="orange"
                        px="8px"
                        py="5px"
```

```
                borderRadius="4px"
              >
                {question.userPosted.charAt(0).toUpperCase()}
              </Avatar>
              <div>{question.userPosted}</div>
            </Link>
          </div>
        </div>
      </div>
    </div>
  </section>
  {question.noOfAnswers !== 0 && (
    <section>
      <h3>{question.noOfAnswers} Answers</h3>
      <DisplayAnswer
        key={question._id}
        question={question}
        handleShare={handleShare}
      />
    </section>
  )}
  <section className="post-ans-container">
    <h3>Your Answer</h3>
    <form
      onSubmit={(e) => {
        handlePostAns(e, question.answer.length);
      }}
    >
      <textarea
        name=""
        id=""
        cols="30"
        rows="10"
        value={Answer}
        onChange={(e) => setAnswer(e.target.value)}
      ></textarea>
      <br />
      <input
        type="submit"
        className="post-ans-btn"
        value="Post Your Answer"
      />
    </form>
    <p>
      Browse other Question tagged
      {question.questionTags.map((tag) => (
        <Link to="/Tags" key={tag} className="ans-tags">
          {" "}
          {tag}{" "}
        </Link>
      ))}{" "}
      or
      <Link
        to="/AskQuestion"
        style={{ textDecoration: "none", color: "#009dff" }}
```

```
                    >
                      {" "}
                      ask your own question.
                    </Link>
                  </p>
                </section>
              </div>
            ))}
        </>
      )}
    </div>
  );
};

export default QuestionsDetails;
```

6. Result and Discussion

- Test Table

Sr. No	Test Case	Input	Expected Result	Actual Result	Status
1	User Registration	Valid name, email, password	User account created	Account stored in database	Pass
2	Duplicate Email	Existing email	Error message	Error displayed correctly	Pass
3	Login	Correct credentials	User logged in	Dashboard displayed	Pass
4	Invalid Login	Wrong password	Error message	Proper error shown	Pass
5	Ask Doubt	Valid title and description	Doubt stored in database	Doubt visible to users	Pass
6	View Doubts	Click doubt	Full details shown	Details displayed	Pass
7	Post Answer	Valid answer	Answer saved	Answer shown	Pass
8	Voting System	Upvote/Downvote	Vote count updated	Updated correctly	Pass
9	Logout	Click logout	Session ended	Redirect to login	Pass
10	Unauthorized Access	Without login	Redirect to login	Working properly	Pass

- Discussion

The DoubtBuddy system was tested for usability, functionality, and reliability. The application successfully handles user authentication, doubt posting, answering, and voting operations.

Key Observations:

- The authentication module provides secure login and registration.
- The database efficiently stores user, doubts, and answers.
- The system handles invalid and unauthorized inputs effectively.
- The frontend and backend integration works seamlessly.
- The application is user-friendly and easy to navigate.

The system provides an efficient solution for knowledge sharing among students and users.

- User Documentation

This section provides instructions for users to operate the DoubtBuddy system.

Registration

1. Open the DoubtBuddy website.
2. Click on the “Register” option.
3. Enter name, email, and password.
4. Click “Submit”.
5. After successful registration, login with your credentials.

Login

1. Enter registered email and password.
2. Click “Login”.
3. The user will be redirected to the dashboard.

Ask a Doubt

1. Click the “Ask Doubt” button.
2. Enter the doubt title and description.
3. Submit the doubt.
4. The doubt will be visible to all users.

View Doubts

1. Navigate to the “All Doubts” section.
2. Click any doubt to view details and answers.

Post an Answer

1. Open a doubt.
2. Enter your answer.
3. Click submit.
4. The answer will appear under the doubt.

Voting System

1. Click the upvote or downvote button.

2. The system updates the vote count.

Logout

1. Click the logout button.

2. The session will be terminated.

7.Conclusion

- **Conclusion**

The DoubtBuddy system was successfully designed and developed using modern web technologies. The application provides a platform where users can share knowledge, solve problems, and collaborate effectively.

The project helped in gaining practical knowledge of:

- Full-stack web development
- Database management
- User authentication
- API development
- Frontend and backend integration

The system achieved its primary objective of creating an interactive discussion platform that allows users to ask doubts and receive answers in an organized manner.

The project demonstrates how technology can be used to improve collaborative learning and communication among users.

- **Limitations of the System**

Despite the successful implementation, the DoubtBuddy system has certain limitations:

1. The system does not provide real-time notifications.
2. Limited user profile customization.
3. Basic search functionality.
4. No email verification or password recovery.
5. No admin moderation panel.
6. Performance is not tested under heavy user load.
7. No mobile application support.

Future enhancements may include:

- Real-time chat and notifications.
- Email verification and password reset.

- Advanced search and filtering.
- Admin control panel.
- AI-based doubt suggestions.
- Mobile and scalable cloud deployment.

REFERENCES

1. MongoDB, *MongoDB Documentation*. Available at:
   https://www.mongodb.com/docs/
   (Accessed: February 2026).

2. Node.js, *Node.js Official Documentation*. Available at:
   https://nodejs.org/en/docs/
   (Accessed: February 2026).

3. Express.js, *Express.js Guide*. Available at:
   https://expressjs.com/
   (Accessed: February 2026).

4. React, *React Official Documentation*. Available at:
   https://react.dev/
   (Accessed: February 2026).

5. Redux, *Redux Documentation*. Available at:
   https://redux.js.org/
   (Accessed: February 2026).

6. JSON Web Token, *JWT Introduction*. Available at:
   https://jwt.io/introduction
   (Accessed: February 2026).

7. GitHub, *Stack Overflow Clone Repository*. Available at:
   https://github.com/Palaksharma23/Stack-Overflow-Clone
   (Accessed: February 2026).

8. Stack Overflow, *Developer Community Platform*. Available at:
   https://stackoverflow.com/
   (Accessed: February 2026).

9. W3Schools, *HTML, CSS, and JavaScript Tutorials*. Available at:
   https://www.w3schools.com/
   (Accessed: February 2026).

10. GeeksforGeeks, *Full Stack Development Resources*. Available at:
    https://www.geeksforgeeks.org/
    (Accessed: February 2026).